Image-to-LaTeX Conversion via CNN-Transformer Architecture

Machine Learning Course Project Report

Mattia Marilli
mattia.marilli@stud.unifi.it
University of Florence
Italy

Marco Trambusti
marco.trambusti@stud.unifi.it
University of Florence
Italy

Abstract

Converting images of mathematical expressions into LaTeX code is a challenging vision-to-language task that requires both accurate symbol recognition and correct understanding of two-dimensional layout. In this project we propose an encoder–decoder approach that combines a convolutional visual encoder with a Transformer-based autoregressive decoder. The encoder extracts a spatial feature representation from the input image, while the decoder generates the corresponding LaTeX token sequence using masked self-attention and cross-attention over the visual features. We evaluate two alternative encoder backbones under the same training and decoding protocol: a standard ResNet-18 and a lightweight custom CNN.

1. Introduction

The automatic conversion of images containing mathematical expressions into symbolic LaTeX markup, commonly referred to as the *image-to-LaTeX* task, has become an important challenge in the intersection of computer vision and natural language processing. Mathematical formulas are inherently structured and exhibit complex spatial layouts, including nested superscripts, subscripts, fractions, and multiline constructs. This combination of visual and symbolic complexity makes the problem a representative instance of *visual sequence transduction*, where models must bridge continuous visual input and discrete, structured output sequences.

Recent advances in deep learning, particularly encoder–decoder architectures with attention mechanisms, offer a promising alternative by jointly modeling visual features and sequence generation. In this paradigm, a visual encoder extracts a representation of the input image, which is then decoded autoregressively into a sequence of LaTeX tokens, allowing for both local and global dependencies to be captured effectively.

In this work, we investigate the impact of different visual encoder designs on image-to-LaTeX transcription quality. Specifically, we compare a task-specific custom CNN encoder with a ResNet-18 backbone, both paired with a Transformer decoder. Through systematic experiments and evaluation with multiple metrics, including Exact Match, BLEU, and normalized edit distance, we aim to highlight how encoder choices influence the model's ability to accurately reproduce mathematical expressions while preserving their structural integrity. Our findings provide insights into architectural considerations for visual sequence transduction and demonstrate the effectiveness of encoder–decoder models for complex cross-modal tasks.

2. Theoretical Background

This work addresses the *image-to-LaTeX* problem, a representative instance of *visual sequence transduction*: given an input image containing a mathematical expression, the goal is to produce a symbolic description in LaTeX. The central challenge lies in bridging two different modalities—continuous visual signals and discrete, structured symbolic language—while preserving the underlying mathematical structure.

2.1. Problem Formulation

Let x denote an input image and $y = (y_1, \ldots, y_T)$ the target token sequence encoding the corresponding LaTeX formula. The task is modeled as conditional sequence generation, where a model learns the distribution:

$$p(y \mid x) = \prod_{t=1}^{T} p(y_t \mid y_{<t}, x), \tag{1}$$

i.e., each token is generated conditioned on the image and on the previously generated tokens.

2.2. Encoder–Decoder Paradigm

A standard approach to conditional sequence generation is the *encoder–decoder* architecture. The encoder maps the

input x into a set of representations (often called *memory*), and the decoder produces the output sequence by repeatedly selecting the next token based on:

- the prefix generated so far (linguistic context),
- the encoder memory (visual context).

This separation is conceptually important: the encoder focuses on extracting informative visual features, while the decoder focuses on modeling the syntax and long-range dependencies typical of mathematical notation.

2.3. Visual Feature Learning with Convolutional Networks

Convolutional Neural Networks (CNNs) are widely used encoders for images because they learn hierarchical representations through local receptive fields. In mathematical expression recognition, CNN encoders are particularly suitable since they capture:

- **local patterns** (strokes, symbols, small subexpressions),
- **spatial layout** (superscripts, subscripts, fraction structure, alignment).

Instead of reducing an image to a single vector, modern architectures often retain a *spatial feature map* which can be interpreted as a sequence (or set) of visual elements. This provides the decoder with fine-grained access to different regions of the expression.

2.4. Sequence Modeling with Transformers

Transformers are sequence models based on attention mechanisms. They process sequences by modeling pairwise interactions between positions, making them effective at capturing long-range dependencies. A key requirement for LaTeX, where opening and closing constructs (e.g., braces, environments) and nested structures can span many tokens.

Masked self-attention and contextualization. Given a partial output sequence, self-attention produces context-aware representations of each position by weighting information from other positions in the sequence. This allows the model to capture structural dependencies, such as matching delimiters, operator precedence patterns, and typical macro usage. In the Transformer decoder, however, a causal mask is applied to the self-attention mechanism to prevent each position from attending to future tokens. As a result, the representation of a token at position (t) can depend only on tokens at positions ($\leq t$). This constraint ensures autoregressive generation, where each new token is predicted based solely on the previously generated tokens and the encoder-provided context.

Positional information. Since attention alone is permutation-invariant, positional encodings are introduced to inject order into the model. This ensures that the decoder can distinguish between the same tokens appearing at different positions and can learn sequential structure.

2.5. Decoding at Inference Time

At inference time, the model must generate y without access to the reference sequence. Since finding the globally optimal sequence is intractable in general, approximate decoding strategies are used:

- **Greedy decoding**, which selects the most probable token at each step, is efficient but can propagate early mistakes.
- **Beam search**, which maintains multiple high-probability hypotheses, better approximates the best overall sequence and is commonly used in structured generation tasks.

3. Implementation

This section describes the implementation of the proposed image-to-LaTeX system, detailing the data pipeline, model components, training procedure, inference, and evaluation.

3.1. Data Pipeline

Dataset loading and splits. The workflow starts by loading the train/validation/test splits from parquet files into Pandas DataFrames from the HuggingFace repository. We base our experiments on the image-to-latex-100k dataset, which provides paired samples of rendered mathematical expression images and their corresponding LaTeX markup. Each row contains (i) an image stored as raw bytes and (ii) the corresponding LaTeX formula as a string.
The split separation is preserved throughout the pipeline: the vocabulary is built *only* from the training split, while validation and test are used exclusively for model selection and final evaluation.

Image decoding and normalization. Images are decoded from bytes using PIL, converted to RGB, and then transformed into tensors through a fixed preprocessing chain. Resizing to a fixed size ensures uniform shapes for batching, while normalization improves optimization stability by controlling the dynamic range of pixel values.

```
transform = T.Compose([
 T.Resize((64, 320)),
 T.ToTensor(),
 T.Normalize((0.5,), (0.5,)),
])
```

Tokenization of LaTeX formulas. Formulas are tokenized using a LaTeX parser that traverses the syntax tree and outputs a flat token list. Tokens include:

- **characters** (letters, digits, operators) excluding whitespace,
- **macros** (e.g., `\frac`, `\sum`),
- **group delimiters** `{` and `}` to preserve explicit grouping structure.

Each sequence is wrapped with `<sos>` and `<eos>` tokens, enabling both teacher forcing during training and a clear stopping criterion at inference time.

Vocabulary construction. A `Vocabulary` component builds a discrete token inventory from the tokenized training formulas. It provides:

- a `stoi` map (token → integer id),
- a `itos` map (integer id → token),
- a `numericalize` method to convert token sequences to id sequences.

Special tokens such as `<pad>` are reserved and used consistently in the loss function and in decoding post-processing.

3.2. Dataset and Batching

Dataset interface. A custom PyTorch `Dataset` exposes samples as `(image_tensor, token_tensor)` pairs. This isolates all per-sample operations (image decoding and token-to-id conversion) behind a standard interface that integrates with `DataLoader`.

Collation and padding. Since formulas have variable length, batching requires padding token sequences to the longest sequence in the mini-batch. The `collate_fn` stacks images into a single tensor of shape `(B, 3, H, W)` and pads captions to `(B, T_max)` with the `<pad>` index.
Padding is a key detail for correctness: all padded positions must be excluded from the loss; otherwise, the model would learn to predict padding rather than meaningful tokens.

3.3. Model Architecture

The model follows an encoder–decoder design:

- **CNN encoder** transforms the image into a sequence of visual features (encoder memory).
- **Transformer decoder** generates LaTeX tokens autoregressively, attending both to previous tokens and to the encoder memory.

3.3.1 CNN Encoder

Two alternative CNN encoders are considered in this work in order to assess how the choice of the visual backbone impacts the final LaTeX transcription quality. In particular, we compare a **ResNet-18** backbone (used as a strong general-purpose feature extractor) against a **custom CNN** designed specifically for this task.

ResNet-18 backbone. The main configuration adopts a ResNet-18 encoder. The output feature map is projected to a target dimension d_{model} (e.g., 256) so that the channel dimension matches the Transformer decoder hidden size.
Rather than collapsing the feature map to a single vector, we preserve spatial information: the projected feature map is flattened across spatial dimensions and interpreted as a sequence of length $H'W'$. This yields a memory tensor with shape `(B, S, d_model)`, where each element can be seen as a "visual token" corresponding to a region of the input image. Preserving spatial structure is crucial for mathematical notation, where meaning depends on layout (e.g., superscripts/subscripts, fraction bars, alignment).

Custom CNN backbone. To complement the ResNet-18 baseline, we also experiment with a lightweight custom CNN encoder. The goal is to test whether a simpler, task-oriented design can retain sufficiently informative spatial features for math-layout understanding while reducing architectural complexity.
The custom encoder follows a block-wise structure with increasing channel capacity ($64 \to 128 \to 256 \to d_{\text{model}}$) and produces, after the final strided stage, a compact feature map that is reshaped into a sequence in the same way as the ResNet-based encoder. This ensures that **the decoder interface is identical** across the two configurations, enabling a fair comparison where only the encoder is changed.

3.3.2 Transformer Decoder

The decoder is implemented as a *stack* of identical Transformer decoder layers. It receives (i) a token prefix (`tgt`) and (ii) the encoder output (`memory`) and produces a distribution over the next token at each position of the prefix.

Input representations. Token ids are first mapped to dense vectors through an embedding matrix $E \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$. Since attention alone does not encode order, a positional encoding is added so that each position t has a distinct representation:

$$\mathbf{z}_t = E(y_t) + \mathbf{p}_t,$$

where in our case $\mathbf{p}_t$ is a learnable positional parameter (truncated to the sequence length).

Residual connections and normalization (why they matter). Each of the three sub-blocks is wrapped with a residual connection (*skip-connection*) and normalization. Residual pathways help gradient flow in deep stacks, while normalization stabilizes optimization by controlling the scale of activations. These mechanisms are one reason why stacking multiple layers is feasible and effective.

Stacking multiple layers (the role of `num_layers`). The full decoder is built by stacking `num_layers` identical decoder layers:

- early layers tend to learn *local* token regularities and coarse alignments to the visual memory,
- deeper layers progressively refine long-range dependencies (e.g., matching braces, nested structures) and sharpen cross-attention alignments to resolve ambiguous symbols or layout-driven constructs (fractions, scripts).

Increasing the number of layers improves representational depth, at the cost of higher computation and potential overfitting if capacity exceeds data support. In this work, `num_layers` is a key hyperparameter controlling decoder depth; specifically, we use 4 layers.

3.4. Training Procedure

Teacher forcing with shifted sequences. Given a padded caption tensor $Y \in \mathbb{N}^{B\times T}$, the model input and targets are created by shifting:

$$Y_{\text{in}} = Y[:, 0 : T-1], \quad Y_{\text{out}} = Y[:, 1 : T].$$

The model predicts logits for each position in Y_{in}, and the loss is computed against Y_{out}.

Loss and padding handling. Cross-entropy loss is used over the vocabulary at each time step. Padded indices are ignored with `ignore_index`. This ensures that sequences of different length contribute correctly without penalizing the model for arbitrary padded positions.

Optimization and stability measures. Adam is used to optimize all parameters jointly. Gradient clipping is applied to avoid exploding gradients and to stabilize training dynamics, especially in early epochs when predictions can be highly uncertain.

3.5. Inference

Stopping criteria. Generation starts from `<sos>` and proceeds token by token until an `<eos>` token is emitted or a maximum length cap is reached to prevent infinite loops in degenerate cases.

Greedy vs. beam search. Two decoders are implemented:

- **Greedy decoding** chooses the argmax token at every step. It is fast but can be brittle: an early mistake may lead to a cascade of errors.
- **Beam search** maintains the best k hypotheses by cumulative log-probability and expands them iteratively. This often improves output quality by trading computation for better global sequence likelihood.

3.6. Post-processing and Canonicalization

Token-to-string conversion. Predicted id sequences are mapped back to tokens via `itos`, and special tokens `<pad>`, `<sos>`, `<eos>` are removed. Remaining tokens are joined into a LaTeX string.

3.7. End-to-End Execution

The main script orchestrates the full pipeline:

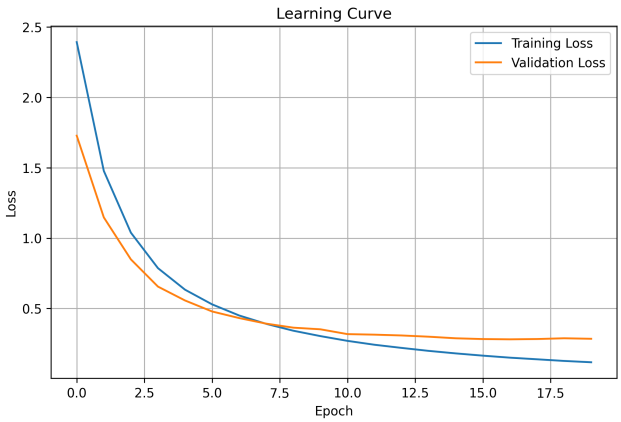
- load data splits and define transforms,
- tokenize formulas and build vocabulary on training data,
- build datasets and data loaders with a padding-aware `collate_fn`,
- initialize the encoder–decoder model,
- train for a fixed number of epochs with validation loss tracking,
- save a learning curve plot,
- evaluate on the test split

Overall, the implementation provides a complete and reproducible workflow from raw dataset entries to quantitative performance reporting.

4. Experiments and Results

This section outlines the experimental setup, the evaluation methodology, and the expectations for the obtained scores. We compare two encoder variants: (i) a **custom CNN** designed for the task and (ii) a **ResNet-18** backbone, both paired with the same Transformer-based decoder and training protocol. The goal is to quantify the impact of the visual encoder choice on LaTeX sequence generation quality.

Epoch selection. All experiments are trained for 15 **epochs**. This choice was guided by an exploratory run of 20 **epochs**, during which we monitored the training and validation loss trajectories. The learning curve shows that after roughly 15 epochs the validation loss improvement becomes marginal, making 15 a reasonable trade-off between convergence and computational cost.



4.1. Experimental Setup

Compared configurations. We evaluate two model configurations under the same conditions:

- **Custom CNN encoder + Transformer decoder**: a lightweight convolutional encoder tailored to preserve spatial information useful for mathematical layouts.
- **ResNet-18 encoder + Transformer decoder**: a stronger generic feature extractor leveraging residual learning, used as a robust baseline.

In both cases, the decoder is kept identical to isolate the contribution of the encoder. Training and evaluation are performed on the same data splits, with the same tokenization strategy and vocabulary.

Inference strategy for evaluation. For reporting test performance, decoding is performed using both greedy decoding and beam search (with a fixed beam width). Greedy decoding provides a simple, fast approximation, while beam search is typically more reliable for structured sequence generation tasks. Evaluating both methods allows comparison of their performance gives a better sense of the most likely overall sequences.

4.2. Evaluation Metrics

Evaluating image-to-LaTeX models is challenging because outputs are discrete strings but the underlying target is a *structured mathematical expression*. To obtain a robust picture of performance, we report complementary metrics.

4.2.1 Exact Match (EM)

Definition. Exact Match measures the fraction of samples for which the predicted string is identical to the reference string (after a normalization / canonicalization step, when applied).
Why it is useful. EM is a strict end-to-end correctness indicator: a match implies the model produced a fully coherent formula without missing or extra tokens. It is particularly meaningful when the downstream usage requires exact LaTeX reproduction or deterministic rendering equivalence.
Limitations. EM is brittle: a single token mismatch (e.g., spacing, an alternative macro, or different bracing) makes the entire sample count as incorrect. For this reason, EM should be interpreted together with graded similarity metrics.

4.2.2 BLEU Score

Definition. BLEU measures n-gram overlap between predicted and reference token sequences .
Why it is useful. BLEU captures *partial correctness*: if the model generates many correct local patterns (e.g., common macro sequences, symbol groups), BLEU can reflect this even when the entire string is not exactly equal. This is helpful for diagnosing incremental improvements when EM remains low.
Limitations. BLEU is not structure-aware; it may not fully reflect mathematical validity (e.g., misplaced braces can break semantics but still share many n-grams).

4.2.3 Normalized Edit Distance

Definition. Normalized edit distance is derived from Levenshtein distance and expresses how many single-character/token edits are needed to transform the prediction into the reference, normalized by sequence length:

$$\text{Score} = 1 - \frac{\text{ED}(\hat{s}, s)}{\max(|\hat{s}|, |s|)}.$$

Why it is useful. This metric provides an interpretable notion of "closeness" and is sensitive to both missing and extra content. Unlike EM, it degrades gradually as errors increase.
Limitations. Similarly to BLEU, edit distance is string-based and does not guarantee semantic equivalence.

Together, this metrics, they reduce the risk of misleading conclusions based on a single criterion.

4.3. Expectations and Interpretation of Scores

Exact Match is expected to be conservative. We do **not** expect very high Exact Match values, even for qualitatively good models, due to the nature of LaTeX representations:

- **Syntactic variability:** the same rendered expression can be written with different (yet valid) LaTeX strings, e.g., alternative macros, optional braces, and spacing commands.
- **Equivalent formatting choices:** minor differences in spacing or style-related commands may not change the final rendering, but EM still counts them as errors.
- **Tokenization sensitivity:** small token-level divergences (e.g., a different grouping strategy) can invalidate EM despite semantic equivalence.

Therefore, EM should be interpreted as a *strict upper-bound indicator* of fully identical transcription, while BLEU and edit-distance based measures are expected to provide a more graded and informative view of progress.

Comparing custom CNN vs ResNet-18. We expect the encoder choice to influence:

- **symbol discrimination** (local visual clarity),
- **layout understanding** (spatial relationships crucial to math),
- **robustness** to style variation and rendering artifacts.

ResNet-18 may provide stronger generic features, whereas the custom CNN may better preserve task-specific spatial cues depending on its design and capacity. The experiments are designed to quantify these trade-offs under identical training and decoding conditions.

4.4. Results and Discussion

Table 1 summarizes the performance of the two encoder configurations under both greedy and beam search decoding. Three complementary metrics are reported: normalized edit distance (ED), Exact Match (EM), and BLEU score.

CNN	Predict Function	ED	EM	BLEU
RN-18	Greedy	88.44%	27.67%	0.7849
RN-18	Beam Search	88.73%	28.16%	0.7889
Custom	Greedy	88.19%	34.35%	0.8094
Custom	Beam Search	88.65%	35.30%	0.8144

Several insights can be drawn from these results:

Impact of encoder choice. The custom CNN consistently outperforms ResNet-18 in terms of Exact Match, while ED is roughly comparable between the two encoders. This suggests that the custom CNN is better at preserving local spatial features and capturing the structural layout of mathematical expressions, which directly benefits token-level correctness and n-gram consistency. ResNet-18, although a stronger generic feature extractor, may abstract away some fine-grained layout details, leading to slightly lower EM.

Greedy vs Beam Search decoding. Beam search consistently improves all metrics for both encoders, although the gains are modest. For instance, the Custom CNN improves from 34.35% EM under greedy decoding to 35.30% with beam search. This aligns with expectations: beam search explores multiple candidate sequences and increases the likelihood of selecting the most accurate LaTeX transcription, particularly in cases where local token predictions might be ambiguous.

Interpretation of metrics. **Exact Match (EM)** remains relatively low, as anticipated, due to the strictness of the metric and the variability in LaTeX representations. Even small differences in braces, spacing, or macro usage can invalidate a match. **BLEU** and **Normalized Edit Distance (ED)** provide a more graded assessment of performance. The custom CNN achieves the highest BLEU score (0.8144) and competitive ED (88.65%), confirming that most predictions are close to the references, even when EM is not perfect.

Overall assessment. The results confirm that encoder design matters for image-to-LaTeX tasks. A task-specific, lightweight convolutional encoder can outperform a more generic ResNet-18 backbone in preserving layout and symbol fidelity, especially when combined with beam search decoding.

5. Conclusions

In summary, our experiments confirm that the encoder–decoder architecture is a well-suited and effective framework for image-to-LaTeX and similar cross-modal sequence generation tasks. When combined with an appropriate visual encoder, this approach allows the model to capture both local visual details and global structural information, leading to accurate and reliable predictions. Overall, these results support the use of encoder–decoder designs as a strong baseline for structured sequence transduction problems.